

Problem Set 1a: Space Cows - Answers

Algorithm Comparison Results

Greedy Algorithm: 6 trips, time taken: 0.000027 seconds

```
[[ 'Betsy'], [ 'Henrietta'], [ 'Herman', 'Maggie'], [ 'Oreo', 'Moo Moo'], [ 'Millie', 'Milkshake', 'Lola'], [ 'Florence']]
```

Brute Force Algorithm: 5 trips, time taken: 0.267630 seconds

```
[[ 'Henrietta'], [ 'Milkshake', 'Oreo', 'Florence'], [ 'Maggie', 'Millie', 'Lola'], [ 'Herman', 'Moo Moo'], [ 'Betsy']]
```

Question 1: Which Algorithm Runs Faster? Why?

Answer:

The **greedy algorithm runs approximately 10,000x faster** (0.000027 vs 0.267630 seconds).

Why:

- **Greedy algorithm:** Has linear/polynomial time complexity. It sorts the cows once $O(n \log n)$ and then iterates through them once $O(n)$, resulting in $O(n \log n)$ overall.
- **Brute force algorithm:** Has exponential time complexity. It must enumerate ALL possible partitions of the cows. For n cows, this is roughly $O(B_n)$ where B_n is the Bell number, which grows extremely fast (Bell numbers grow faster than exponential).

Example: With 10 cows, brute force must check ~115,975 partitions. With 20 cows, it's ~51 billion partitions. This exponential growth explains the dramatic time difference.

Question 2: Does the Greedy Algorithm Return the Optimal Solution? Why/Why Not?

Answer:

No, the greedy algorithm does **NOT** return the optimal solution.

Why Not:

The greedy algorithm returns 6 trips while the optimal solution is 5 trips, proving it's suboptimal.

Root cause - Greedy is myopic: The algorithm always picks the largest cow that fits, but this locally optimal choice doesn't guarantee a globally optimal result. The fundamental issues are:

1. **Poor future planning:** By always taking the largest cow first, it may leave awkward gaps that prevent other cows from fitting together, requiring more trips overall.

2. **No backtracking:** Once a cow is assigned to a trip, it can't be reconsidered. A different ordering might lead to better packing.
3. **Lack of greedy choice property:** Cow transport doesn't have the "greedy choice property" needed for greedy algorithms to work optimally (unlike standard coin change problems).

Analogy: If you're packing boxes and always stuff the largest item first, you might waste space that smaller items could have filled more efficiently if arranged differently.

Question 3: Does the Brute Force Algorithm Return the Optimal Solution? Why/Why Not?

Answer:

Yes, the brute force algorithm **DOES** return the optimal solution.

Why:

1. **Exhaustive enumeration:** The algorithm checks every possible partition of the cows into trips (all valid assignments).
2. **Constraint validation:** It only considers partitions where no trip exceeds the weight limit.
3. **Optimal selection:** It picks the partition with the minimum number of trips among all valid partitions.
4. **Mathematical guarantee:** Since it considers every valid partition, it's mathematically impossible to miss the optimal solution.

Trade-off:

- **Advantage:** Guaranteed optimal (5 trips vs greedy's 6)
 - **Disadvantage:** Much slower (0.267630 seconds vs greedy's 0.000027 seconds)
-

Summary

Aspect	Greedy	Brute Force
Trips Required	6	5 (optimal)
Time Complexity	$O(n \log n)$	$O(B_n)$ - exponential
Actual Runtime	0.000027 sec	0.267630 sec
Optimal?	No	Yes
Practical Use	Large datasets	Small datasets

This is a classic example of the **time vs. optimality trade-off** in algorithm design. Choose based on your constraints: speed or correctness.

Problem Set 1b: Golden Eggs - Dynamic Programming

Problem Description

Given a set of egg weights and a target weight, find the **minimum number of eggs** needed to sum to the target weight. Assumes:

- Infinite supply of each egg weight
- Always have an egg of weight 1 (can always make any weight)

This is equivalent to the classic **unbounded knapsack problem** or **coin change problem**.

Example Test Case

Input: egg_weights = (1, 5, 10, 25), target_weight = 99 **Expected Output:** 9 eggs **Solution:** $3 \times 25 + 2 \times 10 + 4 \times 1 = 99$ **Actual Output:** 9 ✓

Algorithm: Top-Down Dynamic Programming with Memoization

Approach:

```
For each weight, try using each possible egg:  
dp(weight) = min(1 + dp(weight - egg)) for all eggs ≤ weight  
Base case: dp(0) = 0
```

Time Complexity: $O(\text{target_weight} \times \text{num_eggs})$

- Without memoization: $O(\text{branches}^{\text{height}})$ - exponential
- With memoization: $O(\text{target_weight} \times \text{num_eggs})$ - polynomial
- For $n=99$ and 4 eggs: $\sim 99 \times 4 = 396$ operations

Space Complexity: $O(\text{target_weight})$

- Memoization dictionary stores results for all weights from 0 to target_weight

Why Dynamic Programming Works Here:

1. **Optimal Substructure:** The optimal solution for weight W is built from optimal solutions of smaller weights
2. **Overlapping Subproblems:** Many recursive branches compute the same subproblems
 - Without memoization: recalculating $\text{dp}(50)$ happens multiple times
 - With memoization: store result once, reuse many times

Alternative: Bottom-Up DP (Commented in Code)

```
dp = [0] + [float('inf')] * target_weight
```

```

for i in range(1, len(dp)):
    for weight in egg_weights:
        if weight > i:
            break
        dp[i] = min(dp[i], 1 + dp[i - weight])
return dp[target_weight]

```

- **Advantage:** Iterative (no recursion overhead), clearer iteration order
- **Disadvantage:** Always computes all weights 0 to target (even if not needed)
- Both have the same complexity, but bottom-up can be slightly faster in practice

Key Insights:

Aspect	Detail
Problem Type	Unbounded knapsack / Coin change
Greedy Solution	Would use $3 \times 25 + 2 \times 10 + 4 \times 1 = 9$ (happens to be optimal here)
DP Solution	Guaranteed optimal by checking all combinations
Result	9 eggs (optimal)
Time	Fast - polynomial time with memoization